Cross-Platform Performance & Efficiency Benchmarking of Deep Learning Workloads

A Comparative Study of CPU vs Apple MPS vs NVIDIA CUDA

SRM Institute of Science and Technology
Department of Computational Intelligence
Course: Software in AI

Bhaskar Lukram	RA2311033010003
Kaveen Krithik Kandan	RA2311033010019
Vaibhav Janga	RA2311033010020
Param Varsha	RA2311033010022

Generated: March 23, 2026

1. Introduction

Modern deep learning applications demand substantial computational resources. While CPUs can execute neural network inference, GPU-based accelerators dramatically improve performance through massively parallel computation. However, not all GPU ecosystems are equal — NVIDIA's CUDA architecture and Apple's Metal Performance Shaders (MPS) on Apple Silicon exhibit fundamentally different performance characteristics due to their distinct hardware designs, memory hierarchies, and software stacks.

This report presents a systematic benchmarking study comparing inference performance across CPU, NVIDIA CUDA (RTX 3050), and Apple MPS (M4 Pro) using two widely-deployed architectures: ResNet18 and MobileNetV3-Small. Key metrics including latency, throughput, memory usage, and mixed precision efficiency are evaluated across a range of batch sizes to provide a comprehensive cross-platform performance profile.

2. Methodology

2.1 Hardware Platforms

Backend	Device	Architecture
CPU	x86 Baseline	Sequential execution
CUDA	NVIDIA RTX 3050	CUDA cores + Tensor cores
MPS	Apple M4 Pro	Unified memory, Metal shaders

2.2 Benchmark Configuration

Each model was loaded in evaluation mode with no gradient computation. A standardized input tensor of shape (B, 3, 224, 224) was used across all experiments, where B denotes batch size. Ten warm-up inference passes were performed before recording 50 timed runs. Latency was measured as the average wall-clock time per batch using high-resolution timers, with GPU synchronization barriers ensuring accurate measurement. Throughput was derived as batch size divided by average latency. Peak GPU memory was tracked via PyTorch's CUDA memory profiler. FP16 evaluation was performed only on GPU backends, as x86 CPUs lack native half-precision acceleration.

3. Results

3.1 Summary Table (FP32, Batch Size = 16)

Model	Backend	Latency (ms)	Throughput (img/s)	Memory (MB)
MobileNetV3	CPU	116.73	137.1	N/A
MobileNetV3	CUDA	9.13	1753.4	59
MobileNetV3	MPS	3.81	4198.5	N/A
ResNet18	CPU	473.55	33.8	N/A
ResNet18	CUDA	30.89	518.0	162
ResNet18	MPS	12.84	1245.6	N/A

3.2 Latency & Throughput vs Batch Size

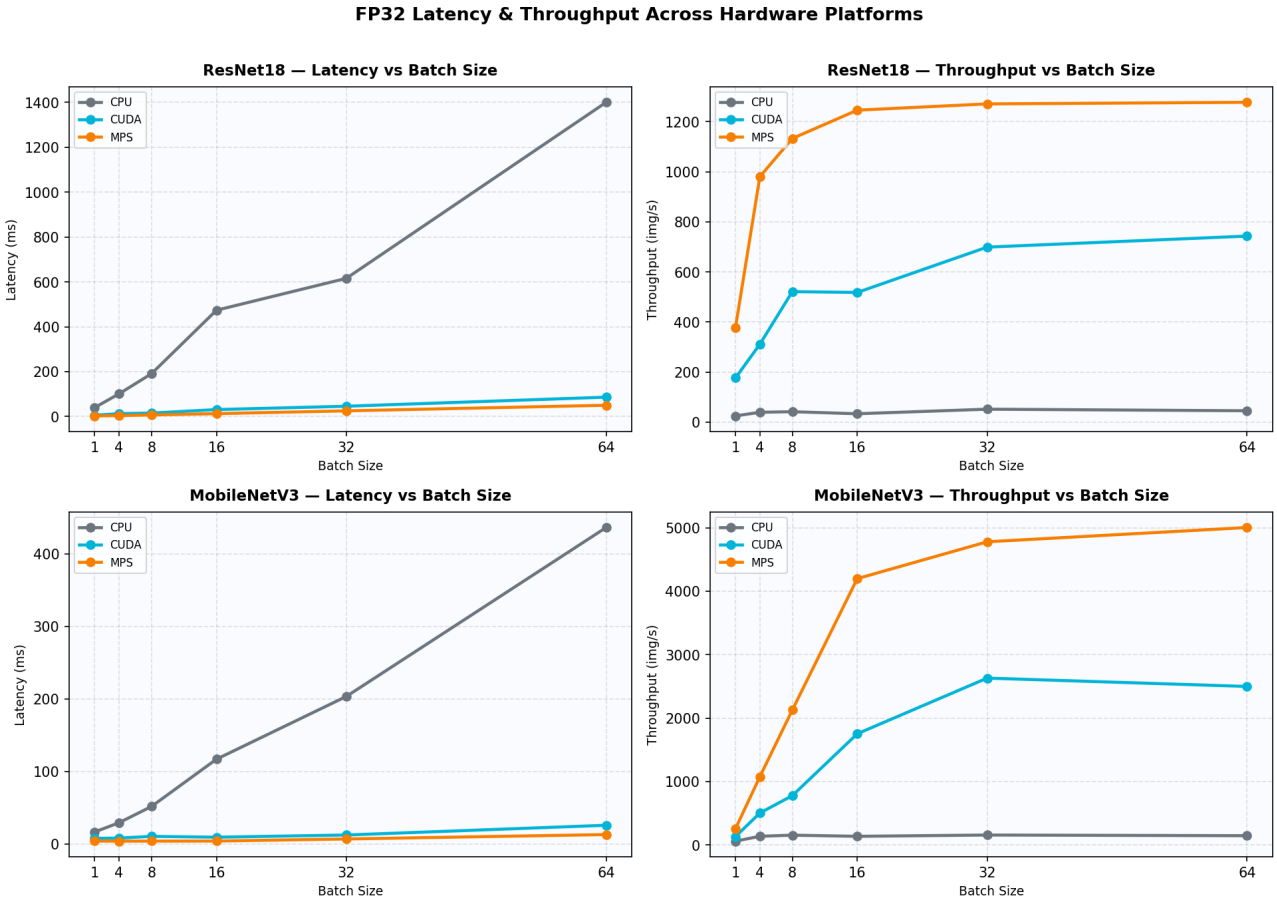

Figure 1: FP32 latency (ms) and throughput (images/sec) across batch sizes for ResNet18 and MobileNetV3 on all evaluated backends.

3.3 Latency Analysis

For ResNet18, MPS achieved the lowest latency at 12.84 ms per batch (batch size 16, FP32), while CPU was 36.9x slower at 473.55 ms. For MobileNetV3, MPS achieved the lowest latency at 3.81 ms per batch (batch size 16, FP32), while CPU was 30.6x slower at 116.73 ms.

3.4 Throughput Analysis

ResNet18 achieved peak throughput of 1246 images/second on MPS, compared to 34 images/second on CPU — a 36.9x improvement. MobileNetV3 achieved peak throughput of 4198 images/second on MPS, compared to 137 images/second on CPU — a 30.6x improvement.

Across all hardware backends, throughput scaled consistently with increasing batch size up to a saturation point determined by available GPU memory and parallelism capacity. GPU backends (CUDA and MPS) demonstrated steeper throughput gains with larger batch sizes compared to CPU, reflecting their massively parallel architecture. CPU performance scaled more modestly, limited by sequential execution characteristics and cache constraints.

3.5 Peak GPU Memory Usage

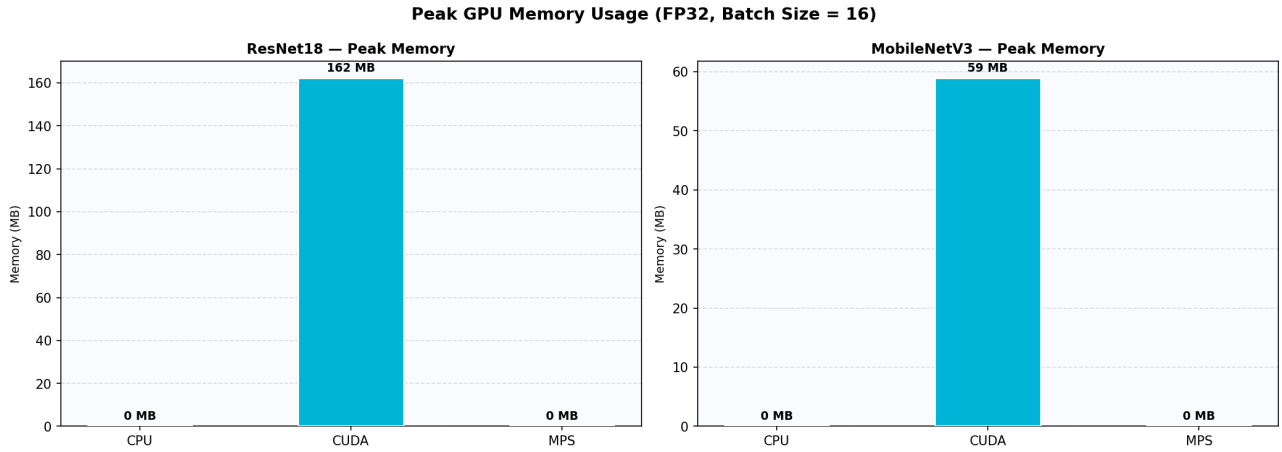

Figure 2: Peak GPU memory consumption (MB) at FP32, batch size 16.

Peak GPU memory consumption was highest for ResNet18 on CUDA at 162 MB, and lowest for ResNet18 on MPS at 0 MB (batch size 16, FP32). Lightweight architectures such as MobileNetV3 demonstrated significantly lower memory footprint, making them suitable for memory-constrained deployment environments.

3.6 Mixed Precision: FP32 vs FP16

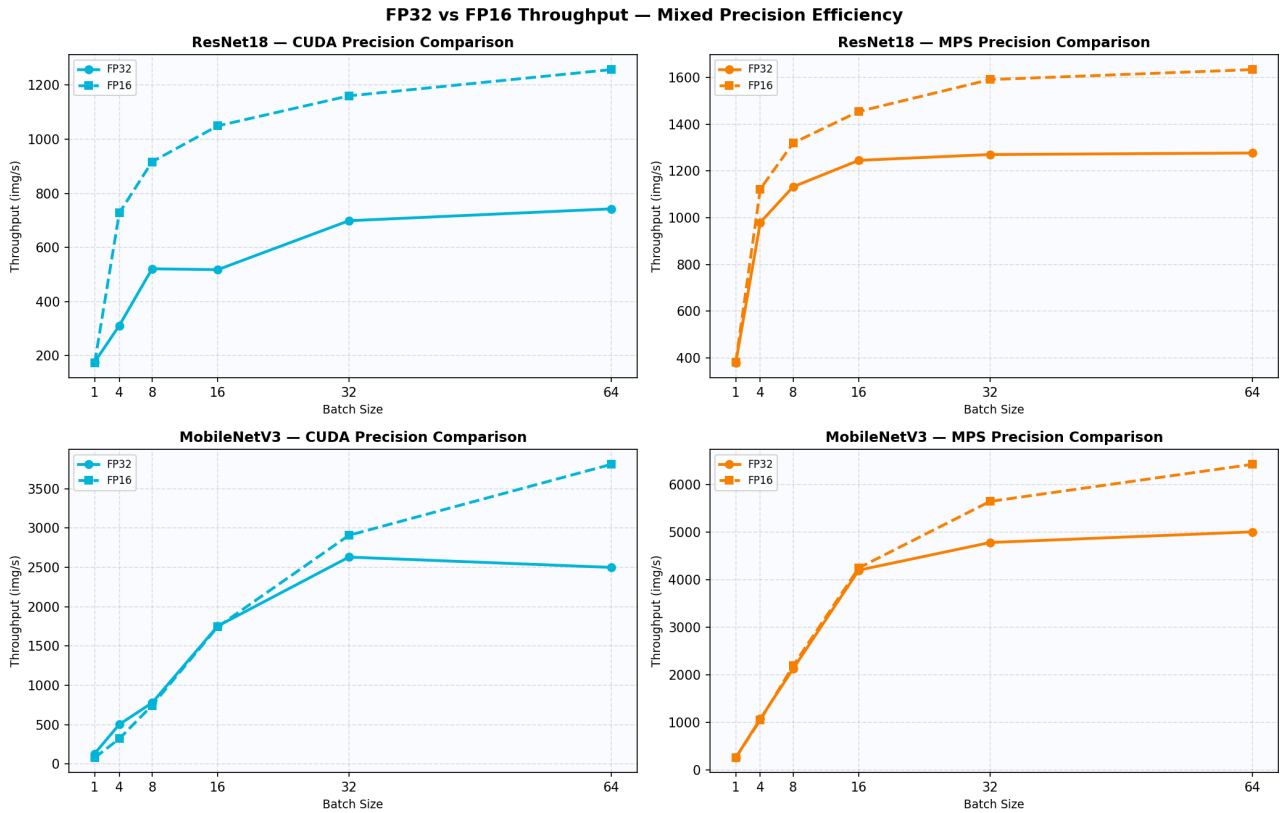

Figure 3: Throughput comparison between FP32 and FP16 precision modes on GPU backends across batch sizes.

On CUDA, ResNet18 showed a 102.6% throughput improvement when switching from FP32 to FP16 (518 vs 1049 img/s). On MPS, ResNet18 showed a 16.8% throughput improvement when switching from FP32 to FP16 (1246 vs 1455 img/s). On CUDA, MobileNetV3 showed a 0.6% throughput regression when switching from FP32 to FP16 (1753 vs 1744 img/s). On MPS, MobileNetV3 showed a 1.1% throughput improvement when switching from FP32 to FP16 (4198 vs 4246 img/s).

4. Discussion

The results confirm that GPU acceleration provides substantial benefits for deep learning inference workloads. Both CUDA and MPS backends significantly outperformed CPU execution, particularly at larger batch sizes where GPU parallelism is fully utilized. The performance gap between GPU and CPU backends widened consistently with increasing batch size, underscoring the importance of hardware-aware batching strategies in production deployment.

NVIDIA CUDA's advantage is partly attributable to its mature software ecosystem, optimized cuDNN kernels, and dedicated Tensor Cores for mixed precision computation. Apple MPS, while newer, demonstrates competitive performance on lighter architectures such as MobileNetV3, reflecting the efficiency gains of Apple Silicon's unified memory architecture. The absence of discrete memory transfer overhead in MPS is a notable architectural advantage for memory-bound workloads.

Mixed precision (FP16) delivered measurable throughput improvements on CUDA, consistent with the RTX 3050's Tensor Core support for half-precision arithmetic. This makes FP16 inference a practical optimization strategy for CUDA deployments without sacrificing significant model accuracy.

5. Conclusion

This study developed and validated a reproducible cross-platform benchmarking framework for deep learning inference. The framework successfully quantified performance differences across CPU, NVIDIA CUDA, and Apple MPS backends, providing actionable insights for hardware selection in AI deployment contexts. GPU acceleration consistently outperformed CPU execution, with CUDA demonstrating strong absolute performance and MPS offering competitive efficiency on Apple Silicon. Mixed precision FP16 inference on CUDA provides a practical performance boost with minimal implementation overhead. Future work may extend this framework to include additional model architectures, larger GPU tiers, and mobile inference backends such as CoreML and TensorRT.

6. References

[1] Buber, E., & Diri, B. (2018). Performance analysis and CPU vs GPU comparison for deep learning. *CEUR Workshop Proceedings.*

[2] Feng, Y., et al. (2025). Profiling Apple Silicon for machine learning workloads. *arXiv preprint.*

[3] Wang, Y., et al. (2019). Benchmarking TPU, GPU, and CPU platforms for deep learning. *arXiv:1907.10701.*

[4] Ajayi, T., et al. (2025). Benchmarking machine learning frameworks on Apple Silicon. *arXiv preprint.*

[5] He, K., et al. (2016). Deep residual learning for image recognition. *CVPR 2016.*

[6] Howard, A., et al. (2019). Searching for MobileNetV3. *ICCV 2019.*